MCMP117 · WEEK 8

Fundamentals of Operating Systems

Week 8 · Midterm Review & Practice

Seven weeks of theory, one exam: what to know, and how to turn it into points.

Electronical Communication Technology Programme · Vocational School · Istanbul Okan University

THE COURSE SO FAR

Where We Are: Half-Way Through

W1
History
batch, multi-programming, time-sharing

W2
OS basics
goals, interrupts, dual mode, structures

W3
Processes
system calls, states, PCB, fork()

W4
Threads
models, Amdahl, scheduling criteria

W5
Scheduling & sync
FCFS … RR, mutex, semaphores

W6
Memory
MMU, fragmentation, paging, TLB

W7
VM & files
page faults, FIFO/OPT/LRU, directories

Theory: Weeks 1–7 — all of it is on the midterm

W8 Midterm exam
today: review and practice

You are here ↑

W9–W14 Hands-on Linux
shell, files, permissions, pipes, processes, scripts

W15 Final exam
whole course

Today's Agenda

1. **The exam** Logistics, what is covered, question types and points.
2. **Recap, Weeks 1–7** One or two slides per week: definitions, diagrams, formulas.
3. **Formula sheet** Every formula of the course on one slide.
4. **Worked problems** Seven problems: every numerical question type of the exam.
5. **Mistakes & strategy** Where points are usually lost, and how to use 90 minutes.
6. **Quiz & questions** Check yourself, then ask anything about the exam.

THE EXAM

Midterm Logistics

When and where

- Date and time: [to be announced]
- Room: [to be announced]
- Duration: [90] minutes
- Arrive 10 minutes early; bring your student ID.

Allowed materials

- Pens; a basic calculator [to be confirmed]
- Closed book: no notes, no phones [to be confirmed]
- Formula sheet provided: [yes / no — to be confirmed]

Format (as in the sample paper)

- Part A: 10 multiple-choice questions × 2 points
- Part B: 4 short-answer questions × 5 points
- Part C: 5 numerical problems, 60 points
- Total: 100 points

Everything in [square brackets] will be confirmed by the instructor on the course Telegram channel. Check it before the exam day.

What Is Covered: Weeks 1–7

Week	Topics	Typical questions
W1	History: batch, multiprogramming, time-sharing, UNIX, Linux	Milestones; U = t_run/(t_run + t_setup), U(n) = 1 − p^n
W2	Definitions, views, goals, interrupts, storage hierarchy, dual mode, OS structures	Explain / compare; mode bit; monolithic vs microkernel
W3	Services, system calls, process, states, PCB, context switch, fork()	State diagram; overhead s/(q + s); count fork() processes
W4	Threads, multithreading models, Amdahl's law, scheduling criteria	Shared vs private; speedup; turnaround, waiting, response
W5	FCFS, SJF, SRTF, Priority, RR; race, critical section, mutex, semaphores	Gantt chart + average waiting time; race outcomes
W6	Binding, MMU, swapping, fits, fragmentation, paging, TLB, segmentation	Fits; address translation; EAT with a TLB
W7	Demand paging, page faults, FIFO/OPT/LRU, thrashing, files, directories	Frame tables; EAT with p; block arithmetic; links

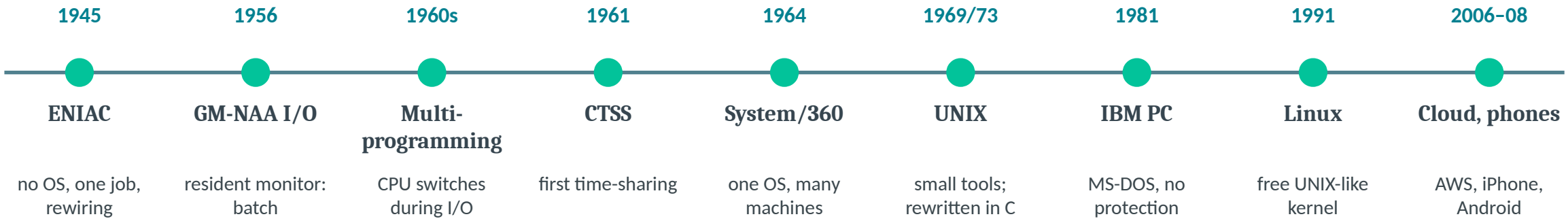
Parts A and B test definitions and explanations from every week; Part C (numerical problems) draws mainly on Weeks 5, 6 and 7.

PART 1

Recap: Weeks 1–7

Definitions, diagrams and formulas you must know — in the words of the course notes.

Why Operating Systems Exist



The story in one line each

- **Batch:** a resident monitor runs jobs back to back — no human set-up time.
- **Multiprogramming:** several jobs in memory; the CPU switches when one waits for I/O.
- **Time-sharing:** short time quanta in rotation; every user feels the whole machine.

```
# CPU utilisation, batch
U = t_run / (t_run + t_setup)

# n jobs, each waiting for I/O a fraction p
U(n) = 1 − pⁿ

# time-sharing: worst-case wait
T_wait ≈ n · q
```

What an OS Is and How It Protects Itself

Definition and views

- Kernel: the one program always running.
- Extended machine (top-down): clean abstractions — files, processes.
- Resource manager (bottom-up): time and space multiplexing.

Goals and components

- Goals: convenience, efficiency, ability to evolve — they conflict.
- Four components: hardware, OS, application programs, users.

Interrupts

- Device signals the CPU; state saved; handler found in the interrupt vector; state restored.
- OS is interrupt-driven; polling wastes CPU.

Storage hierarchy

- Registers ≈ 0.3 ns, cache, RAM ≈ 100 ns, SSD ≈ 50 µs, HDD ≈ 5 ms.
- Registers, cache, RAM are volatile.

Dual mode

- Mode bit 0 = kernel mode, 1 = user mode.
- Privileged instruction in user mode → trap.
- System call: switch to kernel mode and back.

System call cost

- Measured: ≈ 116 ns per system call vs ≈ 1.5 ns per function call.
- Crossing the user/kernel boundary is expensive.

Six Ways to Build an Operating System

Structure	In kernel mode	Speed	Reliability	Example
Simple	everything (no protection)	high	very low	MS-DOS
Monolithic	all OS services	high	medium	UNIX, Linux
Layered	all layers	low	high	THE (1968)
Microkernel	only the tiny kernel	lower	high	MINIX 3, QNX, seL4
Modular	core + loaded modules	high	medium	Linux, Solaris
Hybrid	varies	high	medium–high	Windows, macOS, Android

Microkernel cost of one request: $T_{mono} = 2t_s + t_w$ versus $T_{\mu} = 4t_s + 2t_c + t_w$ (4 mode switches + 2 message copies). Every boundary buys reliability and costs speed.

From printf() to the Kernel and Back



Services

- **For the user:** user interface, program execution, I/O, file-system manipulation, communication, error detection.
- **For the system:** resource allocation, accounting and logging, protection and security.

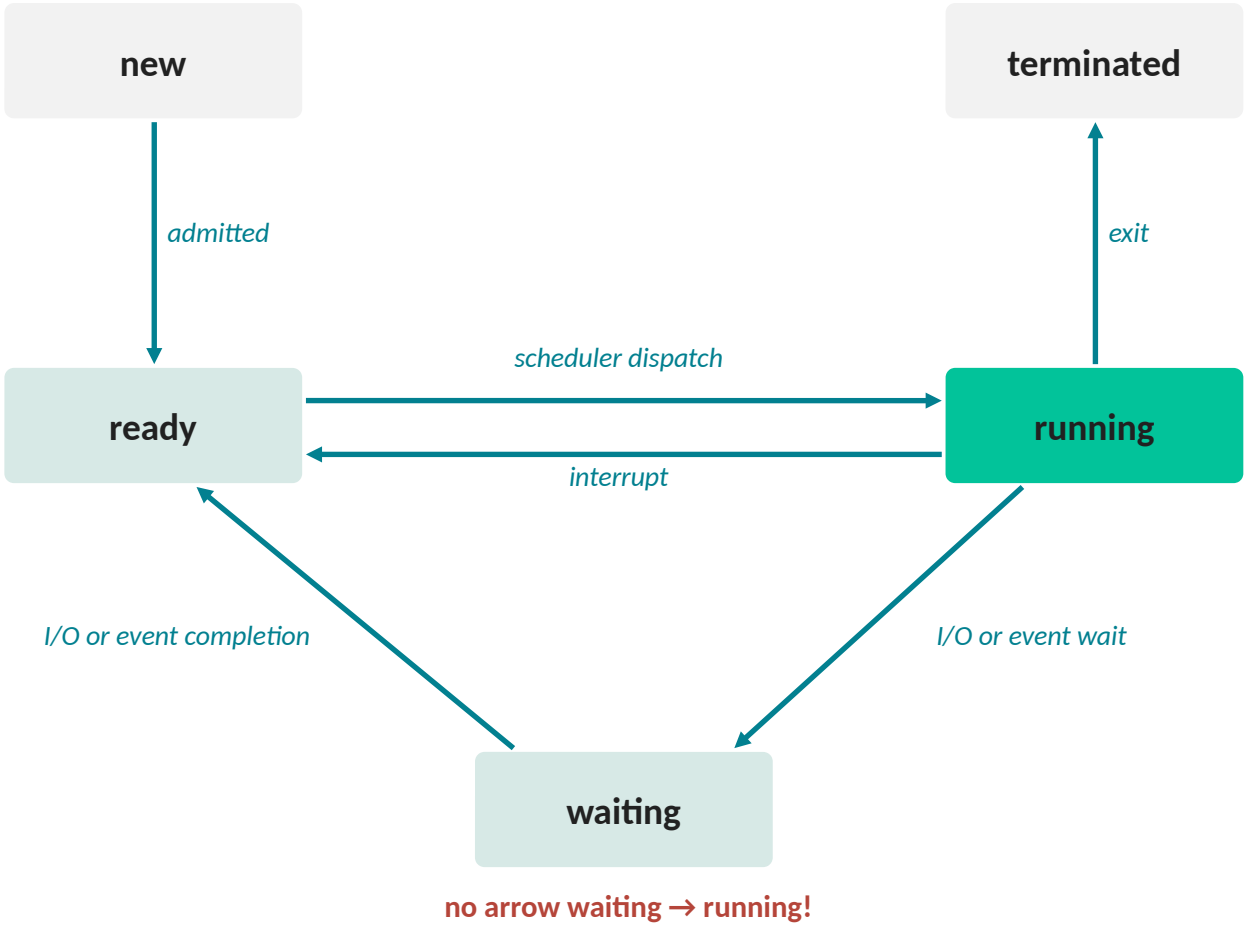
Passing parameters

- 1. In registers (fastest).
- 2. In a memory block; its address in a register.
- 3. On the stack.
- x86-64 Linux: number in rax, up to six parameters in registers.

Six categories

- Process control (fork, exit, wait)
- File management (open, read, close)
- Device management (ioctl)
- Information maintenance (getpid)
- Communication (pipe, socket)
- Protection (chmod)

Process States, PCB, Context Switch, fork()



PCB and context switch

- PCB: state, PID, program counter, registers, scheduling, memory and accounting info, open files.
- Context switch = save state into the old PCB, load the new PCB. Pure overhead: s / (q + s).

Creating processes

- fork() returns 0 to the child, the child's PID to the parent.
- exec() replaces the program; wait() collects the child.
- n forks in a row → 2^n processes. Zombie: ended, not yet waited for.

Threads, Multithreading Models, Amdahl's Law

Shared by all threads	Private to each thread
code (text section)	thread ID
global data and heap	program counter and registers
open files, PID, user ID	stack (local variables)

Benefits of threads

Responsiveness, resource sharing, economy (cheaper than processes: ≈ 4× on our test machine), scalability on multicore CPUs.

Models (user : kernel threads)

- **Many-to-one:** fast, but one blocking call stops all; no parallelism.
- **One-to-one:** Linux and Windows; true parallelism; each thread costs a kernel thread.
- **Many-to-many:** Go goroutines, Java virtual threads.
- Concurrency = progress together; parallelism = at the same instant (needs ≥ 2 cores).

```
# Amdahl's law: serial fraction S, N cores
speedup(N) ≤ 1 / (S + (1 − S)/N)
limit N → ∞:  1 / S

# S = 0.25:
N = 2 → 1.60   N = 4 → 2.29
N = 8 → 2.91   N → ∞ → 4
```

Bursts, Decisions, Dispatcher and Criteria

CPU–I/O burst cycle

CPU-bound: few long CPU bursts. I/O-bound: many short bursts. Most bursts are short.

Four decision points

- 1 running → waiting 2 running → ready
- 3 waiting → ready 4 terminates
- Non-preemptive: only 1 and 4. Preemptive: all four.

Dispatcher

Switch context, switch to user mode, jump to the saved PC. Its time = dispatch latency.

```
T_turnaround = T_completion − T_arrival
T_waiting    = T_turnaround − T_burst
T_response   = T_first run  − T_arrival

CPU utilisation = B / T     (busy / total)
throughput      = n / T

maximise: utilisation, throughput
minimise: turnaround, waiting, response
```

Waiting time = time in the ready queue only. The scheduler cannot change bursts or I/O — only waiting.

Five Algorithms at a Glance

Algorithm	Rule	Preemptive?	Watch out for
FCFS	FIFO ready queue, run to completion	no	convoy effect: short jobs behind a long one
SJF	shortest next CPU burst first; $\tau_{n+1} = \alpha\, t_n + (1 - \alpha)\, \tau_n$	no	optimal average wait (all ready); needs prediction
SRTF	newcomer shorter than the REMAINING time preempts	yes	minimum average wait; long jobs may starve
Priority	highest priority first (smaller number = higher)	either	starvation → cure: aging
Round Robin	quantum q, preempted process to the tail	yes	q too large → FCFS; too small → overhead s/(q + s)

Exam conventions (write them down): context switch = 0 · ties → earlier arrival, then name · smaller number = higher priority · RR: a process arriving when a quantum expires enters the queue BEFORE the preempted one.

Race Conditions, Critical Sections, Semaphores

Race condition

- Several threads access shared data, at least one writes, the result depends on timing.
- counter++ is load – add – store: an update can be lost.

Critical-section problem

- Mutual exclusion: at most one thread inside.
- Progress: no veto from outside, no endless postponement.
- Bounded waiting: no starvation.

Locks

- Disabling interrupts: not on multiprocessors, privileged.
- Peterson (2 threads); atomic test_and_set / CAS.
- Mutex: acquire / release; spinlock vs sleeping.

Semaphores

- wait(S): while (S <= 0); S--; signal(S): S++;
- Binary (= mutex, init 1) or counting (init N).
- Bounded buffer: empty = n, full = 0, mutex = 1.

Deadlock: each thread of a group waits for an event only another member can cause (e.g. wait(S); wait(Q) vs wait(Q); wait(S)). Prevent it: acquire locks in one global order.

Binding, MMU, Allocation and Fragmentation

Address binding

- Compile time: absolute code.
- Load time: relocatable code.
- Execution time: MMU translates every access — all modern systems.

Base and limit

- Logical a: if 0 ≤ a < limit → physical = base + a; otherwise trap.
- Only the kernel may load these registers.

Swapping

- Time = size / transfer rate (100 MB at 50 MB/s: 2 s each way).
- Modern systems swap pages, not processes.

Placement

- First-fit: first hole big enough.
- Best-fit: smallest hole big enough.
- Worst-fit: largest hole.

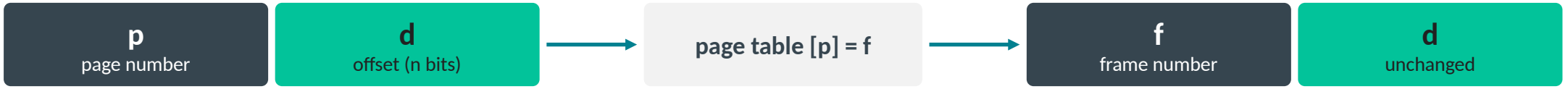
Fragmentation

- External: free memory in holes too small to use (50-% rule: up to 1/3 lost).
- Internal: unused memory inside an allocated block.

Compaction

- Move blocks together into one hole.
- Needs execution-time binding; expensive.
- Better cure: paging.

Pages, Frames, TLB and Segments




```
page size 2ⁿ:  p = ⌊a / 2ⁿ⌋,   d = a mod 2ⁿ
a_phys = f · 2ⁿ + d
4 KB pages + hex: replace the page digit
page-table size = (2ᵐ / 2ⁿ) entries × entry size
32-bit, 4 KB, 4 B → 2²⁰ entries = 4 MB
```

```
# course convention (TLB lookup always counted)
EAT = α(t_TLB + t_m) + (1 − α)(t_TLB + 2t_m)
    = t_TLB + (2 − α) t_m
# simplified (t_TLB ≈ 0):
EAT = α t_m + (1 − α) 2t_m
```

Paging

No external fragmentation; internal: on average half a page per process. Valid–invalid and r/w/x bits checked on every access.

Segmentation

Address ⟨s, d⟩: if 0 ≤ d < limit[s] → base[s] + d, else trap. Variable size → external fragmentation. x86-64 Linux: flat segments, paging does the work.

Demand Paging, Page Faults, Replacement

1 Reference — bit = i → **2 Trap** — MMU → OS → **3 Locate** — page on disk, free frame → **4 Read** — disk → frame → **5 Update** — frame no., bit = v → **6 Restart** — same instruction

```
EAT = (1 − p) t_m + p t_fault
p = page-fault rate; t_fault ≈ ms
→ p must be tiny (≈ 10⁻⁶)
```

Algorithm	Victim	Notes
FIFO	loaded longest ago	cheap; Belady's anomaly possible
OPT	used farthest in the future	minimum faults; needs the future
LRU	used longest ago	good; approximated by the clock

Also know

- Valid–invalid bit; pure demand paging; the pager.
- Copy-on-write: fork() shares pages until a write.
- Dirty bit: a clean victim is not written back.

Thrashing and working set

- Too few frames → constant faults → CPU idle → more processes admitted → worse.
- WS(t, Δ) = distinct pages in the last Δ references; if $D = \Sigma WSS_i > m$ frames → suspend a process.

Files, Access Methods, Directories, Links

File and inode

- Attributes: name, identifier, type, location, size, protection, timestamps.
- Linux: all in the inode — except the name (in the directory).

Operations, tables

- create, open, read, write, seek, close, delete (unlink), truncate.
- open() once → file descriptor; per-process table (file pointer), system-wide table (open count).

```
# direct access, record k
offset = k · R
block  = ⌊k·R / B⌋
in blk = k·R − block · B
check the LAST byte too!
```

Access methods

- **Sequential:** in order (tape model).
- **Direct:** any block n (disk model).
- **Indexed:** search a small index, then jump.

Directory structures

- Single-level → two-level → tree → acyclic graph → general graph (cycles).
- Absolute path from /; relative from the current directory; . and ..

Links

- **Hard:** another name for the same inode; link count.
- **Symbolic:** a file holding a path; dangles if the target is removed.

FORMULA SHEET

Every Formula of Weeks 1–7 on One Slide

Week	Quantity	Formula
1	Batch utilisation / multiprogramming	$U = t_{run} / (t_{run} + t_{setup})$; $U(n) = 1 - p^n$
2	Microkernel vs monolithic (one request)	$T_{mono} = 2t_s + t_w$; $T_{\mu} = 4t_s + 2t_c + t_w$
3, 5	Context-switch (RR) overhead	$s / (q + s)$
3	Processes after n fork() calls in a row	2^n ($2^n - 1$ created)
4	Amdahl's law	speedup $\le 1 / (S + (1 - S)/N)$; limit $1/S$
4	Turnaround and waiting time	$T_{turnaround} = T_{completion} - T_{arrival}$; $T_{waiting} = T_{turnaround} - T_{burst}$
4	Response time	$T_{response} = T_{first\ run} - T_{arrival}$
5	Exponential average of bursts	$\tau_{n+1} = \alpha\, t_n + (1 - \alpha)\, \tau_n$
6	Base and limit	$0 \le a < limit \rightarrow base + a$, else trap
6	Paging	$p = \lfloor a/2^n \rfloor$, $d = a \bmod 2^n$; $a_{phys} = f \cdot 2^n + d$
6	TLB effective access time	$EAT = t_{TLB} + (2 - \alpha)\, t_m$
7	Page-fault effective access time	$EAT = (1 - p)\, t_m + p\, t_{fault}$
7	Direct access, record k	$block = \lfloor kR / B \rfloor$; $offset = kR - block \cdot B$

INTERMISSION

10-Minute Break

Next: seven worked problems — one for every numerical question type.

PART 2

Worked Problems

Try each one yourself first. Every number is checked by code/review_problems.py.

PROBLEM 1 · WEEKS 1, 3, 4

Four Quick Calculations

(a) Multiprogramming (W1)

- Jobs wait for I/O p = 0.6 of the time. U(3)? Smallest n with U ≥ 95 %?
- **U(3) = 1 − 0.6^3 = 78.4 %.** $0.6^5 \approx 0.078 > 0.05$ but $0.6^6 \approx 0.047 \le 0.05 \rightarrow$ n = 6.

(b) Context switch (W3)

- s = 4 µs. Overhead for q = 2 ms and for q = 50 µs?
- **4 / 2004 ≈ 0.20 %;** 4 / 54 ≈ 7.4 %. Tiny quanta waste the CPU.

(c) fork() (W3)

- fork(); fork(); printf("A\n"); fork(); printf("B\n");
- **A: 4 times, B: 8 times,** 8 processes (7 created). Output to a terminal.

(d) Amdahl (W4)

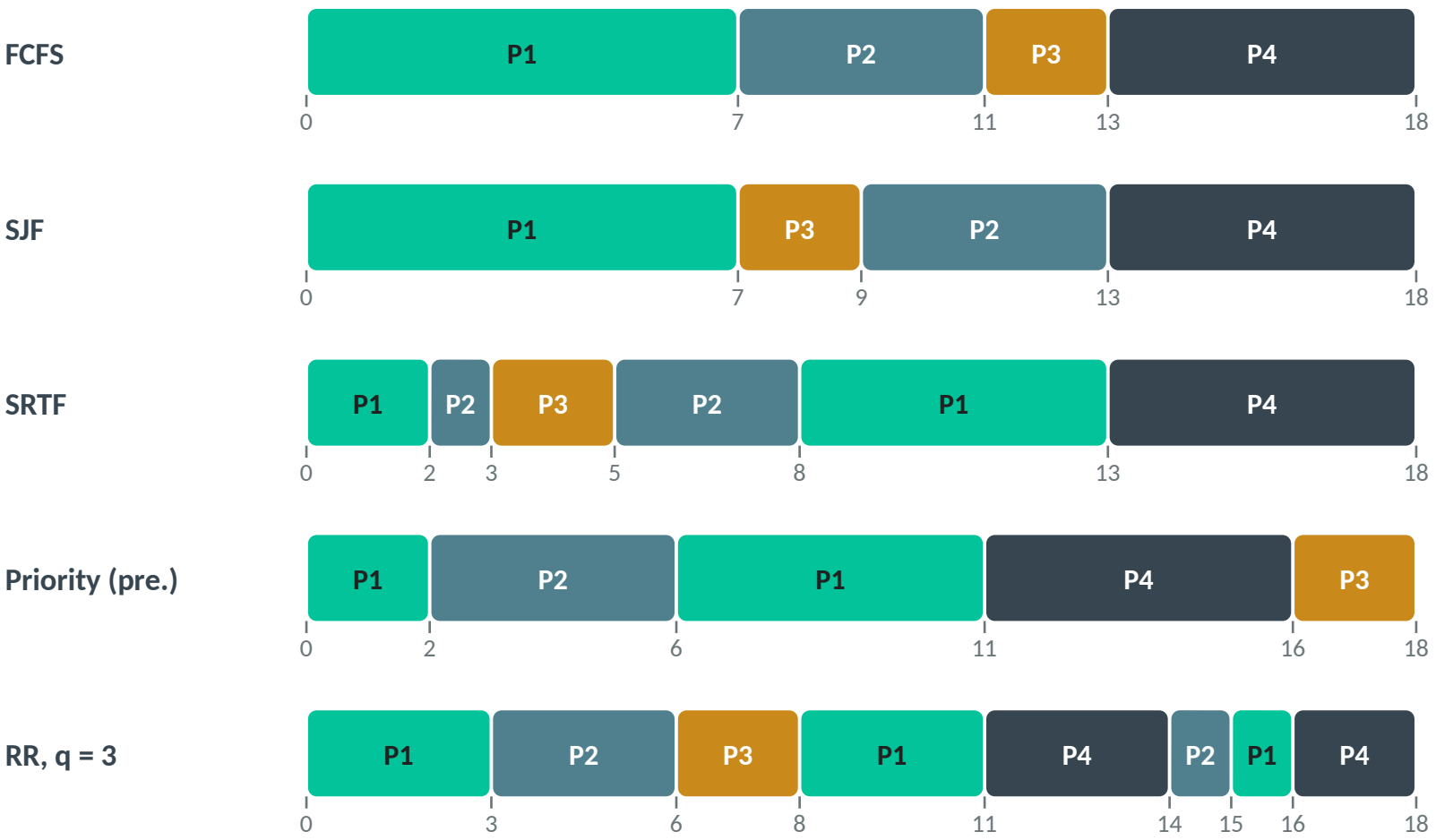
- S = 0.2. Speedup on 4 and 8 cores? Limit?
- **1/(0.2 + 0.8/4) = 2.5;** 1/(0.2 + 0.1) ≈ 3.33; limit 1/0.2 = 5.

One Job Set, Five Schedulers

Process	Arrival	Burst	Priority
P1	0	7	2
P2	2	4	1
P3	3	2	4
P4	5	5	3

Read the chart

- SRTF t = 3: P3 (2) < P2's remaining 3 → preempt.
- SRTF t = 8: P1 and P4 both need 5 → earlier arrival P1.
- RR t = 3: P3 arrives as P1's quantum ends → P3 first.



Results: Averages and Context Switches

Algorithm	Preemptive	Avg waiting	Avg turnaround	Avg response	Switches
FCFS	no	5.25	9.75	5.25	3
SJF	no	4.75	9.25	4.75	3
SRTF	yes	4	8.5	2	5
Priority	yes	5.75	10.25	4.75	4
RR, q = 3	yes	7.25	11.75	2.5	7

```
# SRTF, from the formulas
P1: TAT 13 − 0 = 13, wait 13 − 7 = 6
P2: TAT  8 − 2 = 6, wait 6 − 4 = 2
P3: TAT  5 − 3 = 2, wait 2 − 2 = 0
P4: TAT 18 − 5 = 13, wait 13 − 5 = 8
average waiting = 16/4 = 4 ms
```

What the numbers say

- SRTF has the smallest average waiting time (4 ms) — as the theory promises — but the most context switches after RR.
- Priority serves importance, not averages: low-priority P3 waits 13 ms. RR has the worst average waiting time (7.25 ms) but a good response time (2.5 ms).

A Race and a Semaphore, Step by Step

Step	Thread A (counter++)	Thread B (counter--)	counter
1	load (rax = 10)		10
2		load (rax = 10)	10
3	add 1 (rax = 11)		10
4		add −1 (rax = 9)	10
5	store		11
6		store	9 (should be 10)

Event	S	Result
start (2 printers)	2	
T1 wait	1	T1 enters
T2 wait	0	T2 enters
T3 wait	0	T3 blocks
T1 signal	1 → 0	T3 enters
T2 signal	1	
T3 signal	2	all done

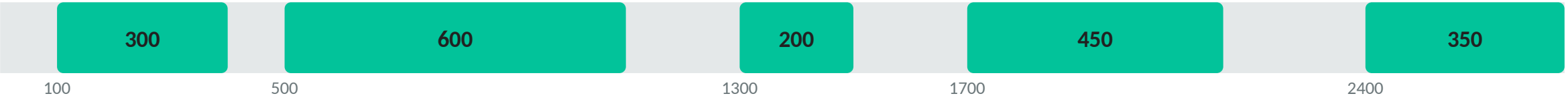
All 20 interleavings of the six steps, checked: counter ends at 9 (9 orders), 10 (2 orders) or 11 (9 orders). Only 2 of 20 orders are correct!

Counting semaphore S = 2: at most two threads use a printer at once.

PROBLEM 4 · WEEK 6

First-Fit, Best-Fit, Worst-Fit

Holes (mint, size in KB) in address order; grey = in use. Requests in order: A = 340, B = 180, C = 420, D = 500 KB.



Request	First-fit	Best-fit	Worst-fit
A = 340	600@500 → leaves 260	350@2400 → leaves 10	600@500 → leaves 260
B = 180	300@100 → leaves 120	200@1300 → leaves 20	450@1700 → leaves 270
C = 420	450@1700 → leaves 30	450@1700 → leaves 30	must wait
D = 500	must wait	600@500 → leaves 100	must wait

First-fit: D waits although 960 KB are free (largest hole 350 KB) — external fragmentation. Best-fit places all four. Worst-fit fails twice: 1380 KB free, largest hole 350 KB. One example proves nothing in general.

PROBLEM 5 · WEEK 6

Address Translation and the TLB

Page	Frame	Valid
0	3	v
1	8	v
2	6	v
3	—	i
4	10 (0xA)	v
5	2	v

16-bit addresses, 4 KB pages: p = first hex digit, d = last three.

```
0x1C3B: p = 1, d = 0xC3B
  frame 8 → 0x8C3B = 35899
  (8 × 4096 + 3131)

0x4F0E: p = 4, d = 0xF0E
  frame 10 → 0xAF0E = 44814

0x2010: p = 2 → 0x6010 = 24592

0x3004: p = 3 is invalid
  → TRAP (no translation)
```

```
t_TLB = 10, t_m = 80 ns, α = 0.95
EAT = 0.95 × 90 + 0.05 × 170
    = 85.5 + 8.5 = 94 ns
check: 10 + 1.05 × 80 = 94
no TLB: 2 × 80 = 160 ns

EAT = 100 ns → α = ?
10 + (2 − α) 80 = 100
α = 0.875
```

Simplified (t_TLB ≈ 0)

0.95 × 80 + 0.05 × 160 = 84 ns. State your convention!

Page Replacement: FIFO and LRU, 3 Frames

Reference string: 1, 2, 0, 4, 2, 3, 2, 1, 2, 4, 0, 2, 0, 1 · 3 frames, initially empty

FIFO

reference	1	2	0	4	2	3	2	1	2	4	0	2	0	1
frame 0	1	1	1	4	4	4	4	1	1	1	1	2	2	2
frame 1		2	2	2	2	3	3	3	3	4	4	4	4	1
frame 2			0	0	0	0	2	2	2	2	0	0	0	0
fault?	F	F	F	F		F	F	F		F	F	F		F

11
faults with FIFO

LRU

reference	1	2	0	4	2	3	2	1	2	4	0	2	0	1
frame 0	1	1	1	4	4	4	4	1	1	1	0	0	0	0
frame 1		2	2	2	2	2	2	2	2	2	2	2	2	2
frame 2			0	0	0	3	3	3	3	4	4	4	4	1
fault?	F	F	F	F		F		F		F	F			F

9
faults with LRU

Page Replacement: OPT and the Comparison

OPT (look forward: replace the page used farthest in the future)

reference	1	2	0	4	2	3	2	1	2	4	0	2	0	1
frame 0	**1**	1	1	1	1	1	1	1	1	1	1	1	1	1
frame 1		**2**	2	2	2	2	2	2	2	2	2	2	2	2
frame 2			**0**	**4**	4	**3**	3	3	3	**4**	**0**	0	0	0
fault?	F	F	F	F		F				F	F			

7
faults with OPT (minimum)



out of 14 references

Checks you can make

- Compulsory faults: 5 distinct pages (0, 1, 2, 3, 4) → at least 5 faults for any algorithm.
- Always OPT ≤ LRU and OPT ≤ FIFO.
- With 4 frames: FIFO 8, LRU 7, OPT 6 — no anomaly here.

Page-Fault EAT and Direct-Access Arithmetic

(a) t_m = 100 ns, t_fault = 6 ms, p = 0.0002

```
t_fault = 6 ms = 6,000,000 ns
EAT = 0.9998 × 100 + 0.0002 × 6,000,000
    = 99.98 + 1,200 = 1299.98 ns
→ about 13 times slower than RAM

slow-down < 5 %:  EAT < 105 ns
100 + p (6,000,000 − 100) < 105
p < 5 / 5,999,900 ≈ 8.3 × 10⁻⁷
```

(b) 120-byte records, 4096-byte blocks: record 750?

```
offset = 750 × 120 = 90,000
block  = ⌊90,000 / 4096⌋ = 21
in blk = 90,000 − 21 × 4096 = 3984

last byte = 90,000 + 119 = 90,119
⌊90,119 / 4096⌋ = 22  → next block!
→ bytes 3984…4095 of block 21
  and 0…7 of block 22: two reads
```

Lesson (a): convert everything to ns first; one fault in 5,000 accesses already makes memory 13 times slower. Lesson (b): always check the last byte of the record — here it crosses a block boundary.

EXAM TECHNIQUE

Where Points Are Usually Lost

Scheduling

- Waiting = turnaround − burst, not 'start time'.
- SRTF compares with the REMAINING time.
- RR: newcomer before the preempted process.

Units

- 8 ms = 8,000,000 ns; 2 ms = 2000 µs.
- KB = 1024 B in paging; MB/s for swap times as given.

Paging

- The offset is never translated.
- Invalid page → trap, not a number.
- a = limit already traps (strict <).

Frame tables

- FIFO: a hit does not change the order.
- LRU: a hit makes the page most recent.
- Count the faults twice.

Definitions

- Concurrency ≠ parallelism.
- Mutual exclusion ≠ progress ≠ bounded waiting.
- Starvation ≠ deadlock.

Answers

- State conventions and units.
- Show the formula before the number.
- Explain 'why' in one sentence.

EXAM TECHNIQUE

Using 90 Minutes Well



≈ 0.9 minutes per point: a 2-point question should not take 10 minutes. [Times assume a 90-minute exam.]

Before the exam

- Redo Problems 1–7 without looking.
- Write the practice exam in the notes in 90 minutes, then mark it.
- Run code/*.py to check your own numbers.

During the exam

- Start with what you know best.
- Draw Gantt charts and frame tables neatly, with times.
- Stuck? Write the formula and move on — come back later.

Partial credit

- Method counts: formula, substitution, result.
- A wrong number with a correct method still earns points.
- Write units on every answer.

QUICK QUIZ

Check Your Understanding

1. A process arrives at t = 3, needs 4 ms of CPU and finishes at t = 15. What are its turnaround and waiting times?

2. With the page table of Problem 5 (page 2 → frame 6), translate the logical address 0x2ABC.

3. t_{TLB} = 10 ns, t_m = 100 ns, hit ratio α = 0.9. Effective access time (course convention)?

4. ln -s a.txt b.txt, then rm a.txt. Can you still read the data through b.txt? Why?

Week 8 — Key Takeaways

- **Weeks 1–4:** history and goals, dual mode and system calls, process states and PCB, threads — know the definitions and one example each.
- **Scheduling:** draw the Gantt chart, then turnaround = completion − arrival and waiting = turnaround − burst; state the conventions.
- **Synchronisation:** counter++ is load–add–store; three requirements; mutex and semaphores; lock order against deadlock.
- **Memory:** base + limit check, fits and fragmentation, $p \mid d$ split, EAT = $t_{TLB} + (2 - \alpha)\, t_m$.
- **Virtual memory and files:** EAT = $(1 - p)\, t_m + p\, t_{fault}$; FIFO / OPT / LRU tables; block = $\lfloor kR/B \rfloor$; hard vs symbolic links.
- **Practice:** the practice exam in the notes and the sample paper — with a clock.

NEXT WEEK

Linux Shell Basics & Filesystem Navigation

Week 9 will cover:

- Getting Linux: lab PCs, WSL (wsl --install) or an Ubuntu VM in VirtualBox
- Terminal vs shell vs kernel — the shell is a system program (Week 2)
- First commands: whoami, date, uname -a, pwd, ls, cd
- Absolute and relative paths in the real directory tree (Week 7)
- Getting help: --help, man, tab completion, history

QUESTIONS?

Discussion

If you have questions after class, ask them on the course Telegram channel — everyone benefits from the answer.

Asst. Prof. Dr. Hasan Oğuz